UNICAP, Curso de Ciências da Computação (C3), simulado
Nome/matrícula:
1._____

Observações:

- Por padrão todos os atributos possuem visibilidade private e os métodos public, crie o construtor e suponha que os gets e sets estão todos IMPLEMENTADOS no modelo padrão.

1. Crie a classe Torre dentro do contexto de um jogo de Tower Defense. Ela deve conter os atributos abaixo. Além disso, crie seu construtor.

codigo - int;

nome - String;

danoBase - double;

inimigosAlcancados – ArrayList<Inimigo> contendo os inimigos que estão atualmente dentro do alcance da torre;

ativa - boolean.

a. Dentro da classe Torre, crie o método atacar(). O método deve percorrer o ArrayList de inimigos alcançados e realizar um ataque em cada um deles. Durante o ataque, o valor de danoBase deve ser subtraído da vida de cada inimigo.
   Após receber o ataque, a situação do inimigo deve ser atualizada de acordo com as seguintes regras:
   - Caso a vida seja maior que 50, a situação deve ser "Resistente";
   - Caso a vida seja maior que 0 e menor ou igual a 50, a situação deve ser "Ferido";
   - Caso a vida seja menor ou igual a 0, a situação deve ser "Eliminado".

2. Crie a classe Inimigo. Ela deve possuir os seguintes atributos e seu respectivo construtor.

- nome - String;

- vida - double;

- situacao - String, podendo assumir os valores "Resistente", "Ferido" ou "Eliminado";

- recompensa - int.

3. Crie a classe TorreElite, que deve herdar da classe Torre. Ela deve possuir um atributo a mais chamado:

- danoCritico - double.

No jogo, as torres de elite utilizam uma mecânica própria de ataque. Também faça seu construtor.

a. Redefina o método atacar().
   Os inimigos cuja situação já seja "Eliminado" não devem sofrer nenhuma alteração.
   Para os demais inimigos:
   - Caso a situação seja "Resistente", o ataque deve utilizar o danoBase;

- Caso a situação seja "Ferido", o ataque deve utilizar o danoCritico.
Após o ataque, atualize novamente a situação do inimigo segundo as regras:
- Vida maior que 50 → "Resistente";
- Vida maior que 0 e menor ou igual a 50 → "Ferido";
- Vida menor ou igual a 0 → "Eliminado".
Ao final, o método deve retornar a quantidade de inimigos eliminados.

4. Sobrecarregue o método atacar() recebendo como parâmetro um ArrayList<Boolean> chamado buffs.

Cada posição do ArrayList<Boolean> corresponde à mesma posição de um inimigo no ArrayList de inimigos alcançados.

Para cada posição:

- Caso o valor seja true, o dano causado naquele inimigo deve ser aumentado em 10;

- Caso o valor seja false, o dano permanece normal.

Os inimigos já eliminados não devem sofrer alterações.

O restante do cálculo deve seguir exatamente as mesmas regras do método atacar() sobrescrito da classe TorreElite.